

DRDO FDR PROJECT — BACKEND REQUIREMENTS

Frontend is already completed using:

- React
- Three.js
- React Three Fiber
- Recharts

Frontend already supports:

- 3D aircraft replay
- telemetry dashboard
- charts
- alerts
- analytics
- replay controls
- CSV-ready architecture

The backend must send data in the EXACT structure expected by the frontend.

1. MOST IMPORTANT REQUIREMENT

Frontend replay system works FRAME BY FRAME.

Backend MUST send telemetry as:

LIST OF FRAMES

Each frame contains:

- timestamp
- aircraft position

- aircraft orientation
- telemetry parameters

2. REQUIRED TELEMETRY FORMAT

Backend response MUST look like this:

```
[  
  {  
    "timestamp": 0,  
  
    "position": {  
      "x": 0,  
      "y": 0,  
      "z": 0  
    },  
  
    "orientation": {  
      "pitch": 5,  
      "roll": 10,  
      "yaw": 2  
    },  
  
    "parameters": {  
      "gps_altitude": 1200,  
      "ground_speed": 320,  
      "engine1_rpm": 3200,  
      "fuel_pressure": 45  
    }  
  }  
]
```

3. WHAT EACH FIELD IS USED FOR

timestamp

→ Used for replay timing and charts

position

→ Used for aircraft movement in 3D scene

orientation

→ Used for aircraft rotation in 3D scene

parameters

→ Used for telemetry dashboard, charts, alerts, analytics

4. IMPORTANT RULE

Backend MUST send:

ARRAY OF FRAMES

GOOD:

```
[  
  {  
    "timestamp": 0,  
    "parameters": {}  
  }  
]
```

BAD:

```
{  
  "speed": [100,200],  
}
```

```
"altitude": [1000,2000]
}
```

Frontend will break if separate parameter arrays are sent.

5. POSITION OBJECT IS MANDATORY

Required for aircraft movement.

Must exist in EVERY frame:

```
"position": {
  "x": 0,
  "y": 0,
  "z": 0
}
```

6. ORIENTATION OBJECT IS MANDATORY

Required for aircraft rotation.

Must exist in EVERY frame:

```
"orientation": {
  "pitch": 5,
  "roll": 10,
  "yaw": 2
}
```

7. PARAMETER NAMING RULE

Frontend uses EXACT parameter names.

Backend MUST use SAME names.

Examples:

CORRECT:

- gps_altitude
- ground_speed
- engine1_rpm
- fuel_pressure

WRONG:

- gpsAltitude
- engineRPM1
- rpm_engine_1

All names must be:

snake_case

8. REQUIRED PARAMETERS

Navigation:

- latitude
- longitude
- gps_altitude
- barometric_altitude
- ground_speed
- true_airspeed

- indicated_airspeed
- vertical_speed
- heading
- magnetic_heading
- track_angle
- wind_speed
- wind_direction

Aircraft Orientation:

- pitch_angle
- roll_angle
- yaw_angle
- angle_of_attack
- side_slip_angle
- bank_rate
- pitch_rate
- yaw_rate

Engine:

- engine1_rpm
- engine2_rpm
- engine1_egt
- engine2_egt
- engine1_oil_pressure
- engine2_oil_pressure
- engine1_fuel_flow
- engine2_fuel_flow
- engine1_vibration
- engine2_vibration
- afterburner_status
- thrust_level

Fuel:

- fuel_tank_left
- fuel_tank_right
- fuel_tank_center
- fuel_consumption_rate
- fuel_temperature
- fuel_pressure
- fuel_transfer_status

Control Surfaces:

- aileron_position
- rudder_position
- elevator_position
- flap_position
- slat_position
- trim_position
- stick_input_x
- stick_input_y
- autopilot_mode

Hydraulics:

- hydraulic_pressure_a
- hydraulic_pressure_b
- hydraulic_temperature
- pump_status
- fluid_level

Electrical:

- battery_voltage
- generator_voltage

- generator_current
- ac_bus_status
- dc_bus_status
- inverter_status

Landing Gear:

- landing_gear_position
- wheel_speed_left
- wheel_speed_right
- brake_pressure
- anti_skid_status

Environment:

- outside_air_temperature
- cabin_pressure
- humidity
- ice_detection_status

Mission:

- radar_mode
- target_lock_status
- weapon_selected
- missile_ready_status

System Health:

- fault_code
- warning_level
- maintenance_flag
- sensor_health
- system_failure_code

9. REQUIRED APIs

1. Upload Flight File

POST /api/upload-flight

Purpose:

Upload CSV or binary FDR file.

Response:

```
{  
  "flight_id": 101  
}
```

2. Get Telemetry Data

GET /api/flight/101/telemetry

Purpose:

Send telemetry frames to frontend.

MOST IMPORTANT API.

3. Get Flight Events

GET /api/flight/101/events

Example response:

```
[  
  {  
    "timestamp": 42,  
    "event": "ENGINE OVERHEAT",  
    "severity": "HIGH"  
  }  
]
```

4. Get Saved Missions

GET /api/flights

Purpose:

Return all uploaded missions.

10. IMPORTANT PROCESSING FLOW

Backend should NOT send raw CSV.

Backend workflow should be:

CSV/Binary File

→ Parse file

→ Validate data

→ Convert into telemetry frames

→ Send JSON response

11. FUTURE SUPPORT

Backend should be designed to support future:

- WebSockets
- live telemetry streaming
- anomaly detection
- real-time alerts
- mission replay
- multi-user access

12. FINAL IMPORTANT NOTE

Frontend replay system STRICTLY depends on this structure:

```
{  
  "timestamp": ...,  
  "position": {},  
  "orientation": {},  
  "parameters": {}  
}
```

If backend changes this structure, frontend replay and telemetry system will break.